การปรับปรุงประสิทธิภาพของระบบสารสนเทศโดยการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้าและการทดสอบแบบอัตโนมัติ

รหัสนิสิต 65313037 นายปิติภัทร กิจสวน

สาขาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์

โครงร่างการนำเสนอ

01 ความเป็นมาและความสำคัญของปัญหา

02 วัตถุประสงค์และขอบเขต

03 แนวคิดและกรอบการวิจัย

04 เครื่องมือและวิธีดำเนินการ

05 ผลการทดลองและอภิปรายผล

06 สรุปผลการวิจัย

1. ความเป็นมาและความสำคัญของปัญหา

- ระบบสารสนเทศขององค์กรมีปัญหาจากข้อมูลนำเข้าที่ไม่ถูกต้อง
- ระบบเดิมไม่มีการทดสอบอัตโนมัติ ทำให้เกิดบัคซ้ำและตรวจสอบยาก
- ทีมพัฒนาใช้เวลามากในการแก้ไขและตรวจโค้ดด้วยตนเอง
- จำเป็นต้องเพิ่มกลไกตรวจสอบข้อมูลและทดสอบแบบอัตโนมัติเพื่อเพิ่มคุณภาพระบบ

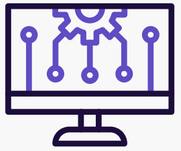
2.1 วัตถุประสงค์ของการวิจัย

1. เพื่อปรับปรุงประสิทธิภาพของระบบสารสนเทศ





2.2 ขอบเขตของการวิจัย



พัฒนา “ระบบลงทะเบียนเรียนต้นแบบ” ด้วย Django Framework

ใช้เป็นระบบจำลองสำหรับการทดสอบ และประเมินคุณภาพซอฟต์แวร์



เพิ่ม Input Validation

ควบคุมรูปแบบและความถูกต้อง ของข้อมูลที่ผู้ใช้กรอกเข้าสู่ระบบ



Automated Dynamic Testing

Pytest → Unit & Integration

Robot Framework → End-to-End



ประเมินคุณภาพโค้ดด้วย SonarQube



ดำเนินการทดสอบใน Local Environment

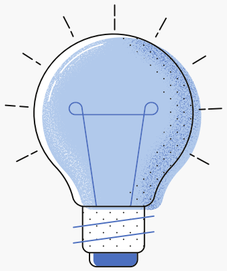
เพื่อควบคุมปัจจัยแวดล้อมให้คงที่



ไม่ครอบคลุม Performance / Security / การเชื่อมต่อระบบจริง

เน้นเฉพาะการปรับปรุงผ่าน Validation และ Automated Testing

3.1 กรอบแนวคิดการวิจัย



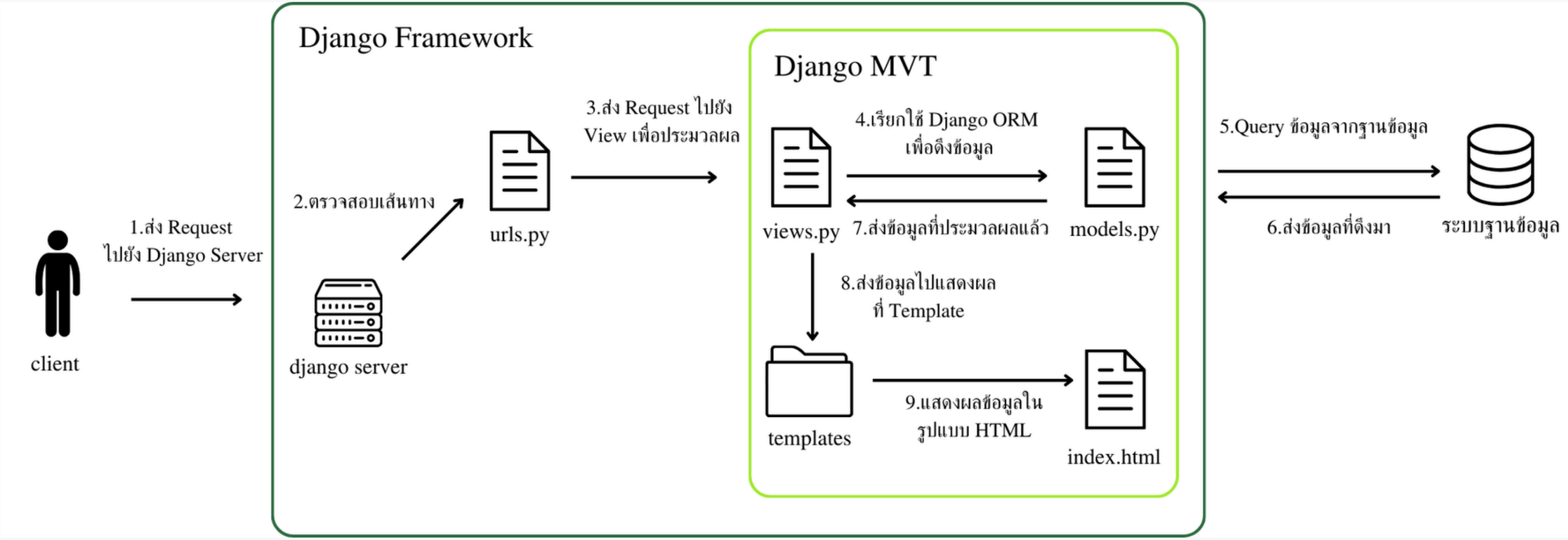
Input	Process	Output
ระบบต้นแบบสองเวอร์ชัน (T0, T1) • T0: ไม่มี Validation/Test • T1: เพิ่ม Validation + Automated Testing	การดำเนินการวิจัย • พัฒนาและรันระบบทั้งสองเวอร์ชันในสภาพแวดล้อมเดียวกัน • ประเมินด้วย SonarQube ตาม 4 Metrics	ผลการปรับปรุงคุณภาพระบบ (T1 > T0) • Reliability ↑ • Maintainability ↑ • Coverage ↑ • Duplications ↓

แนวคิดของการวิจัยมุ่งเน้นการปรับปรุงคุณภาพของระบบสารสนเทศ โดยใช้ระบบต้นแบบ (Prototype) ที่พัฒนาด้วย Django Framework

พัฒนา “สองระบบในสภาพแวดล้อมเดียวกัน” เพื่อเปรียบเทียบผลลัพธ์

โดยใช้เครื่องมือ SonarQube ประเมินคุณภาพซอร์สโค้ด

3.2 สถาปัตยกรรมระบบต้นแบบ



Model เก็บข้อมูลนักศึกษาและรายวิชา

View จัดการตรรกะการลงทะเบียน

Template แสดงผลต่อผู้ใช้

เพิ่ม Validators และ Tests ในแต่ละชั้น

4.1 เครื่องมือที่ใช้



Django → Framework พัฒนาระบบ



Pytest → Unit / Integration Testing



Robot Framework → End-to-End Testing



SonarQube → วิเคราะห์คุณภาพโค้ด

4.2 ขั้นตอนการดำเนินการ

1. วิเคราะห์ปัญหาและโครงสร้างระบบเดิม

2. เพิ่ม Input Validation

3. สร้าง Test (Unit, Integration, E2E)

4. รันทดสอบและเก็บผลทดสอบ

5. ประเมินคุณภาพด้วย SonarQube

6. เปรียบเทียบผลก่อนและหลังปรับปรุง



4.3 เกณฑ์การวัดผล

Reliability

ความน่าเชื่อถือของระบบ



พิจารณาจากจำนวนข้อผิดพลาด
ที่อาจทำให้ระบบล้มเหลว (Runtime Errors)

ค่าน้อยลง → ระบบมีความถูกต้องและเสถียรสูงขึ้น

Maintainability

ความสามารถในการบำรุงรักษาโค้ด



พิจารณาจากโครงสร้างและความซับซ้อนของโค้ด

ค่าน้อยลง → โค้ดสะอาด แก้ไขง่าย
ดูแลระยะยาวได้ดี

Coverage

ความครอบคลุมของการทดสอบ



วัดร้อยละของโค้ดที่ถูกทดสอบ

ค่าสูงขึ้น → ระบบได้รับการทดสอบทั่วถึง

Duplications

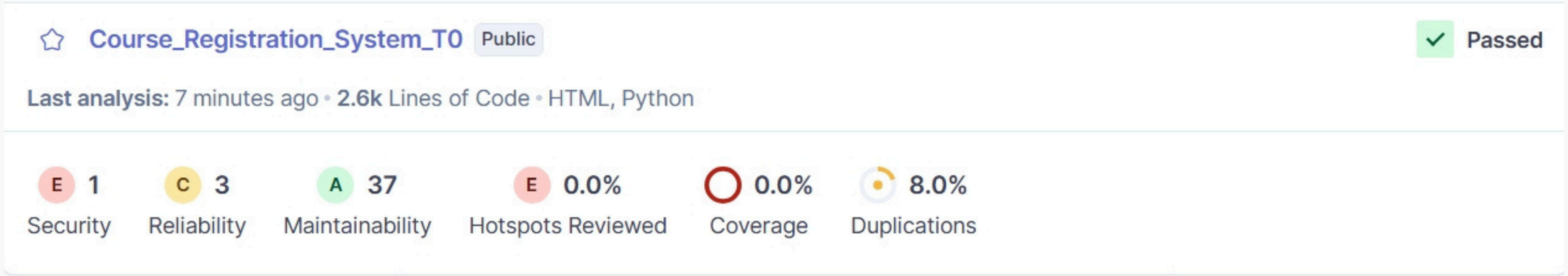
ร้อยละของโค้ดซ้ำซ้อนในระบบ



โค้ดซ้ำมากทำให้ระบบซับซ้อนและดูแลยาก

ค่าน้อยลง → ออกแบบโค้ดมีประสิทธิภาพมากขึ้น

5.1 ผลทดสอบระบบเดิม (T0)



- ระบบเดิม (T0) ยังไม่มีการเพิ่ม Input Validation และ Automated Testing
- พบข้อผิดพลาดเชิงตรรกะ (Reliability = C, Bugs = 3)
- มี Code Smells จำนวนมาก (Maintainability = A, 37 รายการ)
- Coverage = 0% → ไม่มีการทดสอบอัตโนมัติ
- Duplications = 8% → มีโค้ดซ้ำซ้อนในหลายโมดูล



5.2 เพิ่ม Input Validation



เพิ่มกระบวนการตรวจสอบข้อมูล (Input Validation)

เพื่อป้องกันข้อมูลผิดพลาดตั้งแต่ชั้น Form และ Model

ตัวอย่าง

1. แบบฟอร์ม SectionForm → ตรวจสอบหมายเลขกลุ่มเรียน ซ้ำในภาคเรียนเดียวกัน
2. Model Course → ตรวจสอบรูปแบบรหัสวิชา ให้เป็นตัวเลข 6 หลัก

แบบฟอร์ม SectionForm

```
        if section_number and semester and self.course:
            # สร้าง query เพื่อตรวจสอบการซ้ำซ้อน
            qs = Section.objects.filter(
                course=self.course,
                semester=semester,
                section_number=section_number
            )

            # ถ้าเป็นการแก้ไข ไม่นับตัวเองในการตรวจสอบ
            if self.instance.pk:
                qs = qs.exclude(pk=self.instance.pk)

            # ถ้ามีการซ้ำ ให้เพิ่ม error
            if qs.exists():
                self.add_error(
                    'section_number',
                    'กลุ่มเรียน (Sec) หมายเลขนี้มีอยู่แล้วในรายวิชาและภาคเรียนเดียวกัน'
                )
        return cleaned_data
```

Model Course

```
code = models.CharField(
        max_length=10,
        unique=True,
        verbose_name="รหัสวิชา",
        validators=[
            RegexValidator(
                regex=r'^\d{6}$',
                message='รหัสวิชาต้องเป็นตัวเลข 6 หลัก'
            )
        ]
    )
```

5.3 เพิ่ม Automated Testing

เพิ่มกระบวนการทดสอบแบบอัตโนมัติ (Automated Testing)

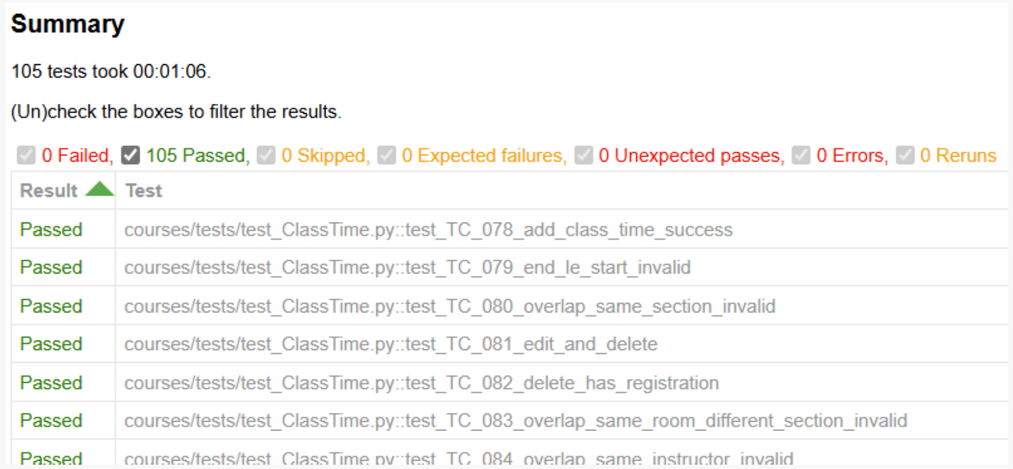
เพื่อประเมินความถูกต้องของระบบจริงโดยไม่ต้องทดสอบด้วยตนเอง แบ่งเป็น 3 ระดับการทดสอบ

1. Unit Test → ตรวจสอบฟังก์ชันย่อยและโมดูลเฉพาะ เครื่องมือที่ใช้ Pytest
2. Integration Test → ตรวจสอบการทำงานร่วมกันของหลายโมดูล เครื่องมือที่ใช้ Pytest
3. End-to-End (E2E) Test → จำลองพฤติกรรมผู้ใช้จริง เครื่องมือที่ใช้ Robot Framework

ผลลัพธ์การทดสอบ 105 Test Cases ผ่านทั้งหมด สำหรับ Pytest

25 Test Cases ผ่านทั้งหมด สำหรับ Robot Framework

Pytest



Summary

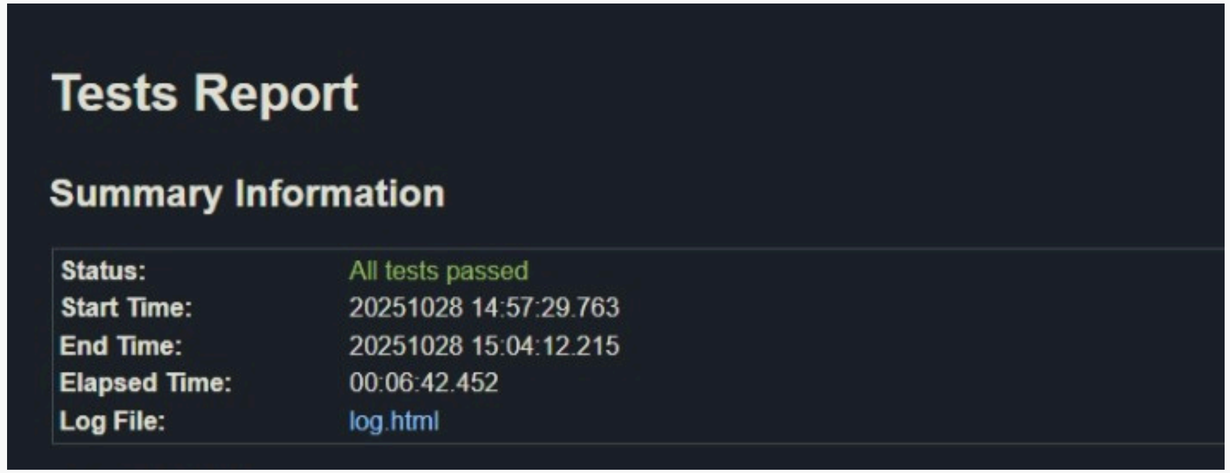
105 tests took 00:01:06.

(Un)check the boxes to filter the results.

0 Failed, 105 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns

Result	Test
Passed	courses/tests/test_ClassTime.py::test_TC_078_add_class_time_success
Passed	courses/tests/test_ClassTime.py::test_TC_079_end_le_start_invalid
Passed	courses/tests/test_ClassTime.py::test_TC_080_overlap_same_section_invalid
Passed	courses/tests/test_ClassTime.py::test_TC_081_edit_and_delete
Passed	courses/tests/test_ClassTime.py::test_TC_082_delete_has_registration
Passed	courses/tests/test_ClassTime.py::test_TC_083_overlap_same_room_different_section_invalid
Passed	courses/tests/test_ClassTime.py::test_TC_084_overlap_same_instructor_invalid

Robot Framework

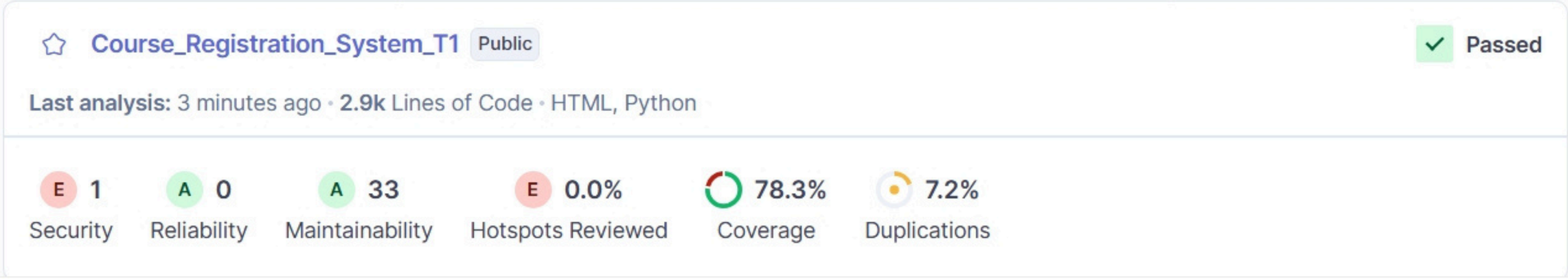


Tests Report

Summary Information

Status:	All tests passed
Start Time:	20251028 14:57:29.763
End Time:	20251028 15:04:12.215
Elapsed Time:	00:06:42.452
Log File:	log.html

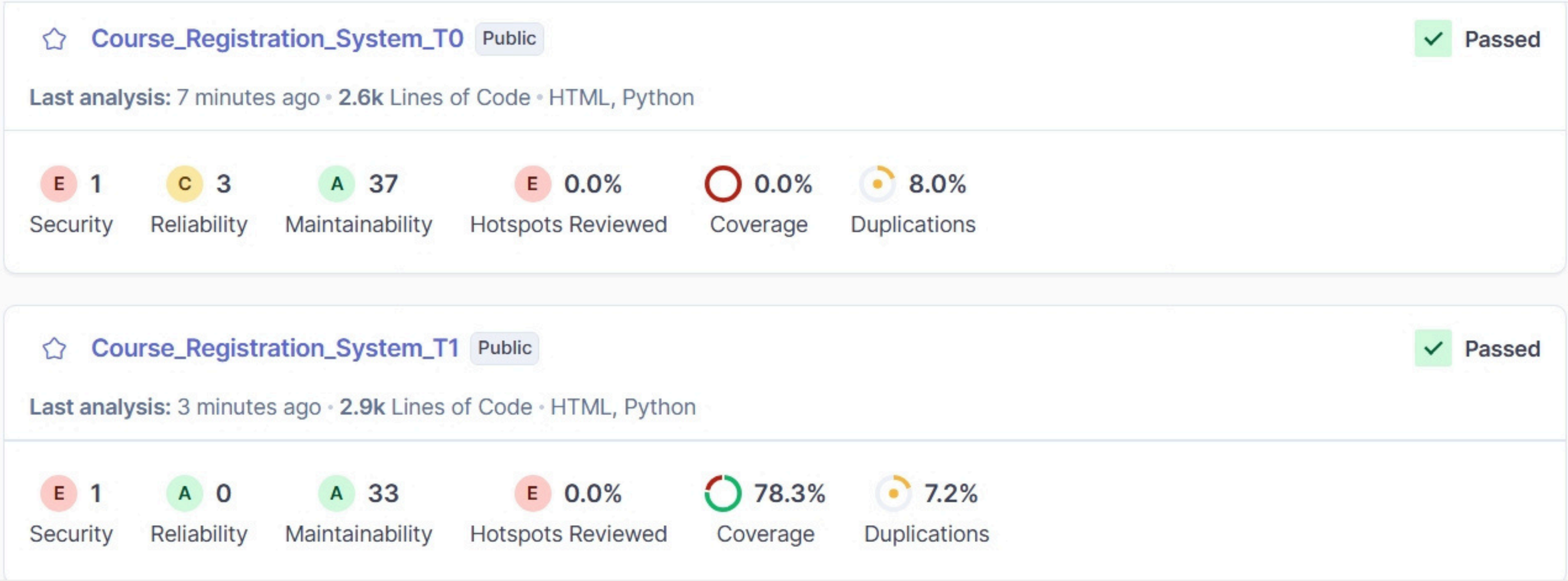
5.4 ผลทดสอบระบบใหม่ (T1)



- ระบบใหม่ (T1) ได้รับการเพิ่ม Input Validation และ Automated Testing
- Reliability = A (0 Bugs) → ลดข้อผิดพลาดทั้งหมด
- Maintainability = A (33 Code Smells) → ลดลงจาก 37 รายการ
- Coverage = 78.3% → เพิ่มจาก 0% ในระบบเดิม
- Duplications = 7.2% → ลดจาก 8.0%



5.5 เปรียบเทียบผลทดสอบ (T0) กับ (T1)



5.5 อภิปรายผล

ผลการปรับปรุงระบบแสดงให้เห็นว่า การเพิ่ม Input Validation และ Automated Testing ช่วยลดข้อผิดพลาดของข้อมูลและเพิ่มความน่าเชื่อถือของระบบ

- ระบบใหม่ (T1) มีค่า Reliability ดีขึ้นจากระดับ C → A
  และ Coverage เพิ่มจาก 0% → 78.3%
  แสดงถึงความครอบคลุมของการทดสอบที่สูงขึ้นอย่างมีนัยสำคัญ
- Maintainability ลดลงจาก 37 → 33 รายการ
  แสดงว่าโค้ดมีความสะอาดและปรับปรุงได้ง่ายขึ้น
  สอดคล้องกับหลักการ Clean Code และ Testing Pyramid
- Duplications ลดลงจาก 8.0% → 7.2%
  แสดงถึงการออกแบบโค้ดที่มีประสิทธิภาพมากขึ้น
- ผลลัพธ์สอดคล้องกับแนวคิดของ Humble & Farley (2010) ที่ระบุว่า
  การทดสอบอัตโนมัติช่วยลดข้อผิดพลาดและเพิ่มความเสถียรในการส่งมอบระบบ



6.1 สรุปผลการวิจัย

งานวิจัยนี้มุ่งปรับปรุงคุณภาพระบบสารสนเทศ
ด้วยการเพิ่ม Input Validation และ Automated Testing

ใช้ “ระบบลงทะเบียนเรียน (Django Framework)” เป็นระบบต้นแบบ
→ พัฒนา 2 เวอร์ชัน
T0 = ระบบเดิม | T1 = ระบบใหม่ที่เพิ่ม Input Validation + Automated Testing

ผลการประเมินจาก SonarQube
- Reliability: C → A (Bugs 3 → 0)
- Maintainability: A (Smells 37 → 33)
- Coverage: 0% → 78.3%
- Duplications: 8.0% → 7.2%
- Security: 1 รายการ (คงที่)



6.1 สรุปผลการวิจัย

- ระบบใหม่มีความถูกต้อง เสถียร และคุณภาพโค้ดดีขึ้น
  → ลดข้อผิดพลาด เพิ่มความครอบคลุมของการทดสอบ
- Django สนับสนุนการตรวจสอบข้อมูลนำเข้า
  ด้วย clean() และ validators
  → ลดความผิดพลาดของข้อมูลได้อย่างมีนัยสำคัญ
- ยืนยันว่า Input Validation + Automated Testing
  เป็นแนวทางที่มีประสิทธิผลในการยกระดับคุณภาพซอฟต์แวร์



6.2 ข้อเสนอแนะในอนาคต



1. ขยายขอบเขตการทดสอบ
เพิ่ม Performance & Security Testing
ใช้ Apache JMeter, OWASP ZAP, Bandit
เพื่อประเมินความเร็วและตรวจหาช่องโหว่ของระบบ



2. ผสานแนวคิด CI/CD
เชื่อมต่อระบบทดสอบกับ Pipeline อัตโนมัติ
ใช้ Jenkins / GitHub Actions / GitLab CI
เพื่อตรวจสอบคุณภาพโค้ดทุกครั้งที่ commit



3. เพิ่มความซับซ้อนของระบบต้นแบบ
ขยายระบบให้ใกล้เคียงระบบจริงมากขึ้น
เพิ่มโมดูล, API ภายนอก, และระบบผู้ใช้หลายระดับ



4. เปรียบเทียบเครื่องมือทางเลือก
ศึกษาเครื่องมือทดสอบและวิเคราะห์อื่น
Semgrep, SonarLint, Cypress, Playwright
เพื่อประเมินประสิทธิภาพและความเหมาะสม



5. ประเมินจากผู้ใช้งานจริง
ทำ User Acceptance Testing (UAT)
เก็บข้อมูลจากผู้ใช้เพื่อวัดความพึงพอใจ
และประสบการณ์ใช้งาน



6. พัฒนาเป็น Framework / Guideline
ยกระดับแนวทางเป็นมาตรฐานองค์กร
สร้าง Quality Assurance Framework
สำหรับ Django/Python